

UNIVERSIDADE PAULISTA – UNIP

**BRUNO FERREIRA DE ROLIM SANTOS – RA H77IDE9
ERICK JONATHAN OLIVEIRA DOS SANTOS – RA R0464
VICTOR HUGO SANTOS MACHADO – RA R6717D2
WESLEY DE SOUZA – RA H781JJ0**

**PROJETO INTEGRADO MULTIDISCIPLINAR III (PIM III)
PLATAFORMA EDUCA TECH – SISTEMA SAAI**

São Paulo
2026

BRUNO FERREIRA DE ROLIM SANTOS – RA H77IDE9
ERICK JONATHAN OLIVEIRA DOS SANTOS – RA R0464
VICTOR HUGO SANTOS MACHADO – RA R6717D2
WESLEY DE SOUZA – RA H781JJ0

PROJETO INTEGRADO MULTIDISCIPLINAR III (PIM III)
PLATAFORMA EDUCA TECH – SISTEMA SAAI

Projeto Integrado Multidisciplinar III
apresentado ao Curso Superior de
Tecnologia em Análise e Desenvolvimento de
Sistemas da Universidade Paulista – UNIP
para a obtenção do título de tecnólogo.

São Paulo
2026

RESUMO

Este relatório técnico multidisciplinar apresenta o planejamento, a documentação estrutural e o desenvolvimento do sistema SAAI (Sistema de Avaliação e Aprendizado Integrado) para a empresa fictícia EducaTech Solutions. O ecossistema foi projetado de forma sinérgica integrando engenharia de software ágil, modelagem lógica com diagramas UML, arquitetura baseada no paradigma de Orientação a Objetos, persistência relacional com Microsoft SQL Server, além de prototipação de interfaces UX/UI funcionais em total conformidade com as Heurísticas de Usabilidade de Nielsen, segurança com tratamento de chaves criptografadas no contexto da LGPD e práticas sustentáveis aplicadas da computação verde. O resultado consolida uma infraestrutura computacional modular, escalável e segura voltada para a gestão de treinamentos e avaliações online de excelência.

Palavras-chaves: Automação Educacional. Engenharia de Software. Banco de Dados Relacional. Interface Responsiva. Sustentabilidade Computacional.

Abstract

This multidisciplinary technical report presents the planning, structural documentation, and development of the SAAI system (Integrated Evaluation and Learning System) for the fictitious company EducaTech Solutions. The ecosystem was synergistically designed by integrating agile software engineering, logical modeling with UML diagrams, an object-oriented architecture, relational persistence with Microsoft SQL Server, and functional UX/UI interface prototyping in full compliance with Nielsen's Usability Heuristics. Furthermore, it incorporates security measures featuring encrypted key management within the context of the LGPD (General Data Protection Law) and sustainable practices applied to green computing. The final outcome consolidates a modular, scalable, and secure computational infrastructure tailored for high-quality online training and evaluation management.

Keywords: Educational Automation. Software Engineering. Relational Database. Responsive Interface. Computing Sustainability.

SUMÁRIO

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO DO NEGÓCIO	5
1.1 Objetivos do Projeto	5
1.2 A Empresa Fictícia	6
2 ENGENHARIA DE SOFTWARE: REQUISITOS E FUNCIONAMENTO	7
2.1 Requisitos Funcionais (RF)	7
2.2 Requisitos Não Funcionais (RNF)	8
2.3 Detalhamento dos Perfis de Acesso	9
3 MODELAGEM UML E ARQUITETURA DE SOFTWARE	10
3.1 Diagrama de Casos de Uso	11
3.2 Diagrama de Classes	12
3.3 Diagrama de Sequência	13
4 MODELAGEM E IMPLEMENTAÇÃO DO BANCO DE DADOS (SQL)	15
4.1 Entidades, Atributos e Lógica MER	15
4.2 Script de Implementação DDL	16
4.3 Consultas SQL (DML) para Auditoria	17
5 INTERFACE WEB RESPONSIVA E PROTOTIPAÇÃO UX/UI	18
5.1 Estrutura Semântica (HTML5)	19
5.2 Prototipação Interativa e Regras de Usabilidade	20
5.3 Responsividade dinamica.....	21
5.4 Prototipação Interativa e Regras de Usabilidade.....	22
6 SISTEMA ACADEMICO	23
6.1-6.7 Estruturas.....	24-31
7 CONCLUSÃO	32
REFERÊNCIAS	33

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO DO NEGÓCIO

Este relatório técnico multidisciplinar constitui o Projeto Integrado Multidisciplinar (PIM III) para o curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Paulista (UNIP). O propósito geral e central deste trabalho acadêmico de engenharia consiste no planejamento completo, na documentação estrutural, na modelagem lógica e no desenvolvimento técnico de uma plataforma corporativa e educacional baseada na web. A aplicação foi inteiramente projetada para atender aos requisitos operacionais e de expansão da empresa fictícia EducaTech Solutions, que atua ativamente no setor de tecnologia educacional (EdTech). O sistema foi oficialmente batizado de SAAI, acrônimo para Sistema de Avaliação e Aprendizado Integrado.

O ecossistema do software SAAI foi concebido para unificar de forma prática, sinérgica e aplicada os conceitos de engenharia de software ágil, modelagem de dados relacionais e normalização, arquitetura e programação orientada a objetos (POO) com tratamento de segurança, desenvolvimento de interfaces front-end responsivas e dinâmicas, além da incorporação obrigatória de práticas sustentáveis alinhadas com a legislação ambiental vigente. Diante do aumento exponencial da demanda por cursos e qualificações na modalidade de ensino a distância (EAD) e treinamentos corporativos assíncronos, a EducaTech Solutions identificou a necessidade crítica de possuir um software proprietário robusto que reduzisse a dependência de plataformas de terceiros de código fechado.

1.1 Objetivos do Projeto

- **Objetivo Geral:** Projetar, documentar, modelar e implementar uma plataforma digital educacional e corporativa robusta e segura, capaz de gerenciar com precisão usuários complexos, perfis com diferentes níveis de acesso, bancos de questões de múltipla escolha, gerenciamento de matrículas, aplicação de provas cronometradas e a respectiva emissão de certificados válidos apoiada por um banco de dados relacional sólido.

- **Objetivo Lógico:** Estruturar logicamente toda a arquitetura abstrata do ecossistema educacional através de modelagem avançada baseada no Modelo Entidade-Relacionamento (MER) e diagramas estruturais e comportamentais da linguagem UML (Casos de Uso, Classes e Sequência), aplicando os conceitos de herança e encapsulamento, garantindo a integridade referencial e mitigando gargalos e falhas de processo antes do início da codificação.

- **Objetivo Técnico:** Implementar o banco de dados utilizando SQL Server, aplicando chaves primárias, chaves estrangeiras, consultas SQL, procedures e boas práticas de desenvolvimento de banco de dados, além de construir uma interface web responsiva utilizando HTML5 semântico estrutural e CSS3 fluido com design tokens, com foco em usabilidade (Heurísticas de Nielsen) e total conformidade com os regulamentos da LGPD.

	Objetivos	Específicos:
•	- Aplicar conceitos de modelagem MER de forma normalizada;	
	- Desenvolver relacionamentos complexos entre tabelas e assegurar restrições de chaves;	
	- Garantir a segurança física dos dados e aplicar conceitos de LGPD por meio do tratamento de chaves hash criptografadas;	
	- Otimizar o desempenho de consultas utilizando Joins e estruturas limpas.	

1.2 A Empresa Fictícia

A empresa parceira EducaTech Solutions atua no segmento de EdTech desenvolvendo soluções sob medida para instituições acadêmicas e ecossistemas corporativos focados em treinamento de pessoal. Sua missão central fundamenta-se em democratizar o acesso à qualificação contínua e ao ensino por meio de sistemas escaláveis, performantes e intuitivos.

2 ENGENHARIA DE SOFTWARE: REQUISITOS E FUNCIONAMENTO

O levantamento de requisitos seguiu metodologias ágeis de engenharia de software para mapear as reais necessidades do negócio, dividindo o escopo do sistema

em duas grandes frentes técnicas: os requisitos funcionais que descrevem o comportamento explícito do sistema e os requisitos não funcionais que garantem a qualidade e a segurança geral do produto de software.

2.1 Requisitos Funcionais (RF)

- RF01 – Gerenciamento Geral de Usuários: O sistema deve permitir o cadastro, edição, consulta e exclusão lógica de usuários vinculados ao ecossistema acadêmico, exigindo campos obrigatórios como nome completo, e-mail único, CPF validado, telefone e atribuição de papel.
- RF02 – Autenticação e Controle Hierárquico de Perfis: O mecanismo de login deve validar as credenciais do usuário comparando o e-mail e a senha digitada com chaves hash criptografadas no banco de dados. O sistema deve barrar ou liberar funcionalidades de forma dinâmica com base em 4 níveis de acesso: Administrador (ADM), Professor, Coordenador e Aluno.
- RF03 – Cadastro de Questionários e Categorização: Módulo exclusivo para que os professores possam criar e gerenciar bancos de questões de múltipla escolha. Cada questão deve possuir um enunciado em formato texto completo, alternativas de 'A' a 'E', indicação da alternativa correta e vínculo obrigatório a uma categoria ou disciplina.
- RF04 – Aplicação de Avaliações em Tempo Real (Quiz): O sistema deve permitir que os alunos iniciem avaliações baseadas nos bancos de questões cadastrados. A tela deve apresentar um cronômetro com contagem regressiva ativa (tempo limite). Ao finalizar, a correção e o cálculo da nota final do estudante devem ser executados de forma totalmente automatizada pela aplicação.
- RF05 – Painel de Controle e Relatórios Acadêmicos: Interface analítica voltada para Coordenadores e Professores, permitindo o cruzamento de dados de desempenho por turmas, médias globais de notas, ranking de engajamento e acompanhamento individual da evolução pedagógica dos alunos.
- RF06 – Emissão Automática de Certificados: Após a conclusão da prova com nota igual ou superior à média de aprovação estabelecida, o sistema deve gerar automaticamente um registro de certificado, incluindo data de emissão, nome do aluno, curso e um código alfanumérico único e exclusivo para verificação de autenticidade

externa.

- RF07 – Gerenciamento Operacional de Matrículas: O sistema deve conter funções específicas para realizar a matrícula do aluno em um curso e disciplina, além de permitir o cancelamento e a alteração do status da matrícula de forma segura.

2.2 Requisitos Não Funcionais (RNF)

- RNF01 – Orientação a Objetos no Back-End: Toda a lógica de negócios, controladores e camadas de acesso devem ser obrigatoriamente codificados na linguagem C#, utilizando os pilares da POO como herança, polimorfismo, encapsulamento e abstração de classes.
- RNF02 – Persistência de Dados Relacional: O armazenamento dos dados operacionais e históricos do sistema deve ser estruturado sob um banco de dados relacional utilizando o Microsoft SQL Server, assegurando conformidade com as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade).
- RNF03 – Interface Web Fluida e Responsiva: O design de interface do front-end deve ser desenvolvido com HTML5 semântico estrutural e folhas de estilo CSS3 puras, garantindo adaptação dinâmica e layout fluido para computadores desktop, notebooks, tablets e smartphones sem distorção de layout.
- RNF04 – Segurança, Privacidade e LGPD: O sistema deve estar em conformidade total com as diretrizes da Lei Geral de Proteção de Dados (Lei nº 13.709/2018). As senhas dos usuários nunca devem ser gravadas em texto limpo, exigindo o uso de algoritmos de hashing seguros (como BCrypt ou SHA-256) antes da gravação física.
- RNF05 – Desempenho e Concorrência: A arquitetura deve suportar múltiplas requisições assíncronas e transações simultâneas de dados, mantendo um tempo de resposta de carregamento de páginas inferior a 2 segundos em condições normais de uso da rede acadêmica.

2.3 Detalhamento dos Perfis de Acesso

1. Administrador (Idm): Controle geral de disciplinas e dados do sistema, suporte operacional e infraestrutura de logs.
2. Coordenador: Gestão de cursos, alocação de disciplinas, análise de relatórios analíticos para decisões pedagógicas e aprovação de matrículas.

3. Assistente Educacional / Professor: Suporte operacional de retaguarda, registro de alunos, controle de frequência, emissão de declarações, alimentação do banco de questões e acompanhamento de turmas.
4. Aluno: Realização de matrículas, solicitações de cancelamento, consulta de horários, acesso ao conteúdo de disciplinas, realização de questionários, provas cronometradas e emissão de notas/certificados.

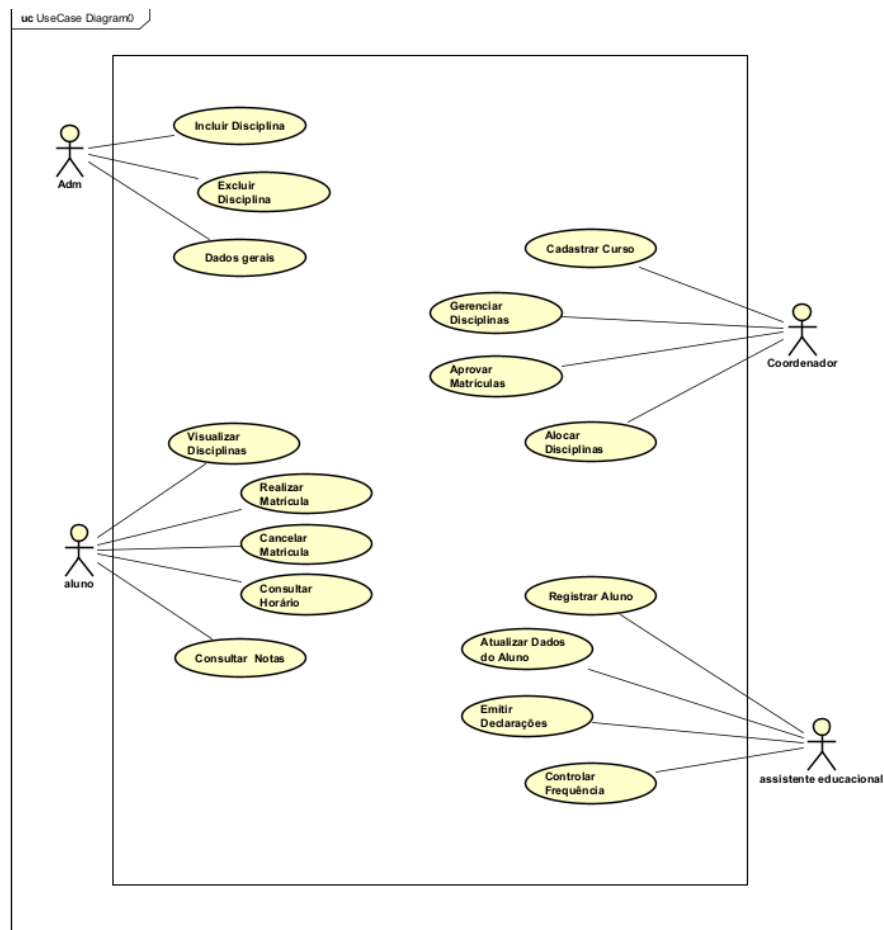
3 MODELAGEM UML E ARQUITETURA DE SOFTWARE

A arquitetura lógica do sistema foi documentada por meio da Linguagem de Modelagem Unificada (UML) utilizando o software Astah UML. O processo de modelagem visual permitiu converter as regras de negócio abstratas em componentes lógicos estruturados de código.

3.1 Diagrama de Casos de Uso

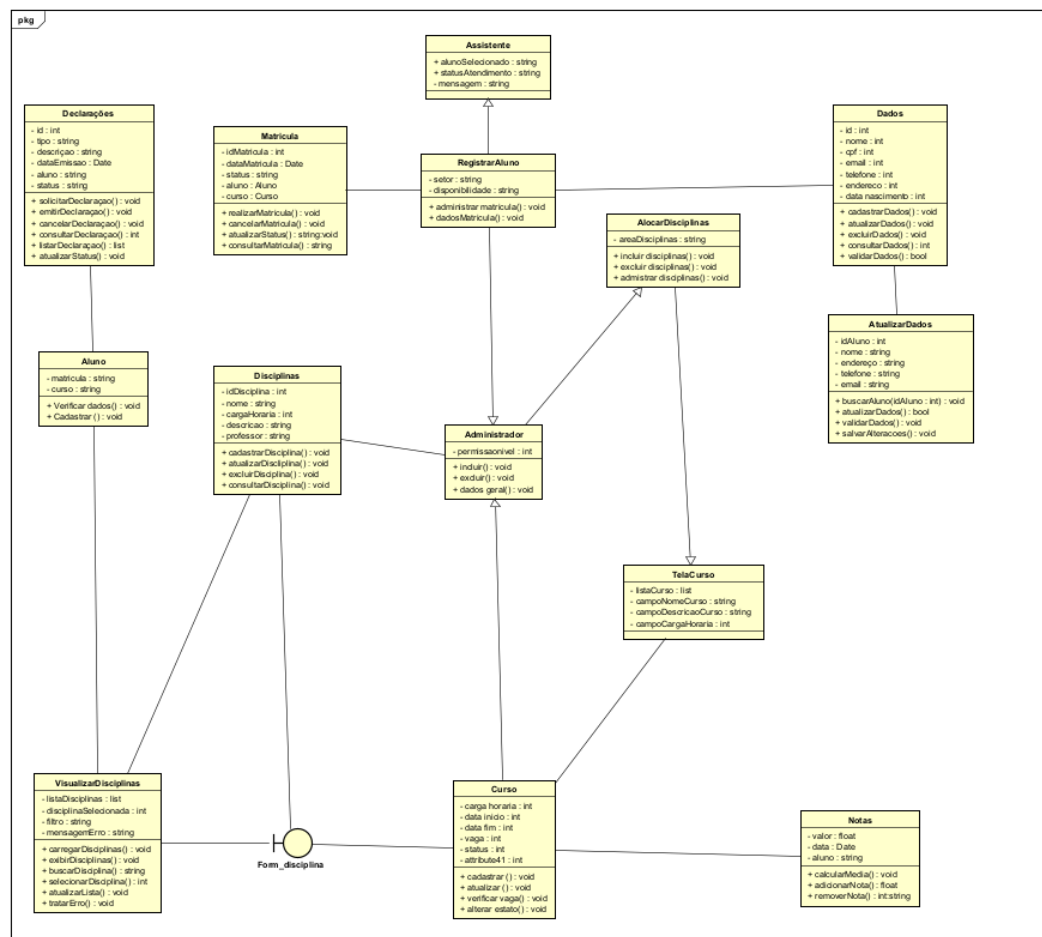
O desenvolvimento no Astah UML seguiu três passos fundamentais para delimitar o escopo:

1. Fronteira do Sistema: Criação do limite físico e lógico para delimitar o escopo da plataforma SAAI.
2. Atores: Posicionados externamente à fronteira, representando os quatro perfis identificados (Administrador, Coordenador, Professor/Assistente e Aluno).
3. Casos de Uso: Inclusão das ações do sistema (ex: Realizar Matrícula, Cadastrar Curso, Incluir Disciplina, Cadastrar Questão, Responder Quiz), conectando cada actor às suas respectivas permissões por meio de linhas de associação direta.



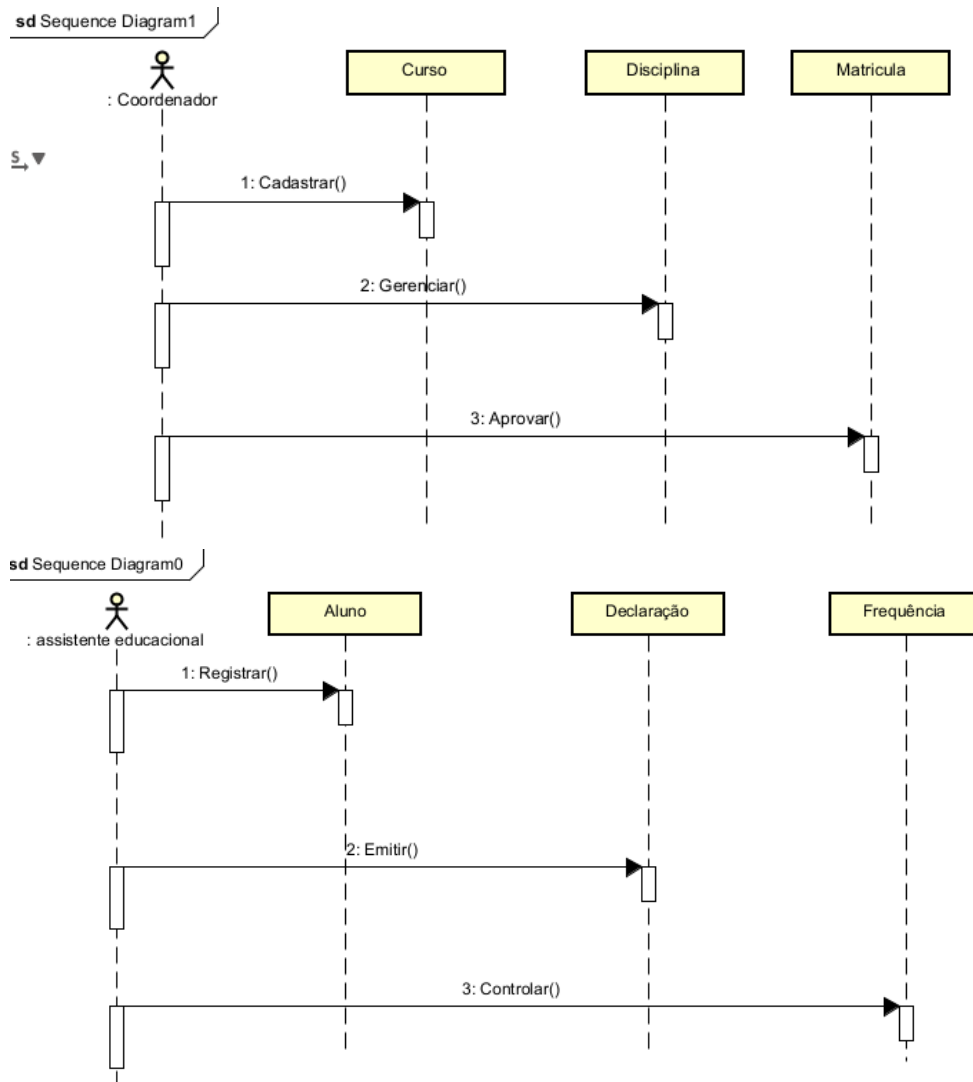
3.2 Diagrama de Classes

Para estruturar a base de dados e a lógica do sistema, mapeamos as entidades centrais (Aluno, Curso, Disciplina, Matrícula) com suas características e comportamentos. Por exemplo, a classe Matrícula contém atributos como idMatrícula e status, e métodos como realizarMatrícula() e cancelarMatrícula(). Foram aplicadas associações diretas (ex: Curso relacionado a Notas), herança para propriedades dos perfis e dependências operacionais. Visualmente, as classes principais foram concentradas no centro do diagrama, deixando as telas de suporte nas laterais.

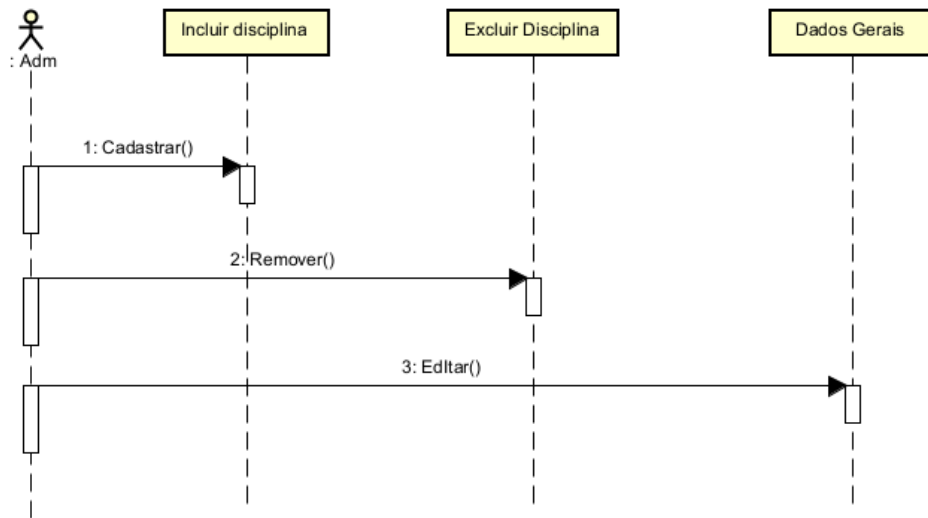


3.3 Diagrama de Sequência

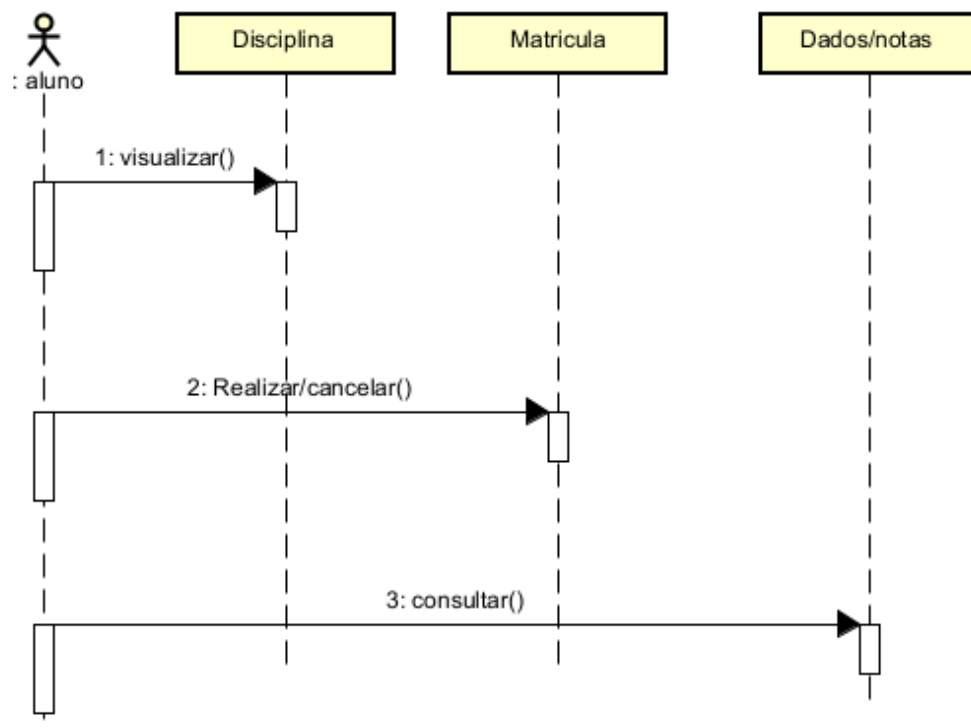
Este diagrama foi essencial para entender a linha do tempo e a troca de mensagens entre os envolvidos nas operações principais. Utilizou atores, linhas de vida (tempo de atividade), barras de ativação e mensagens síncronas. O processo simulado demonstrou a interação em cadeia do sistema: o Administrador inicia a ação chamando o método Cadastrar() no Coordenador; este aciona o Assistente Educacional via Adicionar(), que por fim aplica as regras diretamente ao Aluno com o método Aplicar().



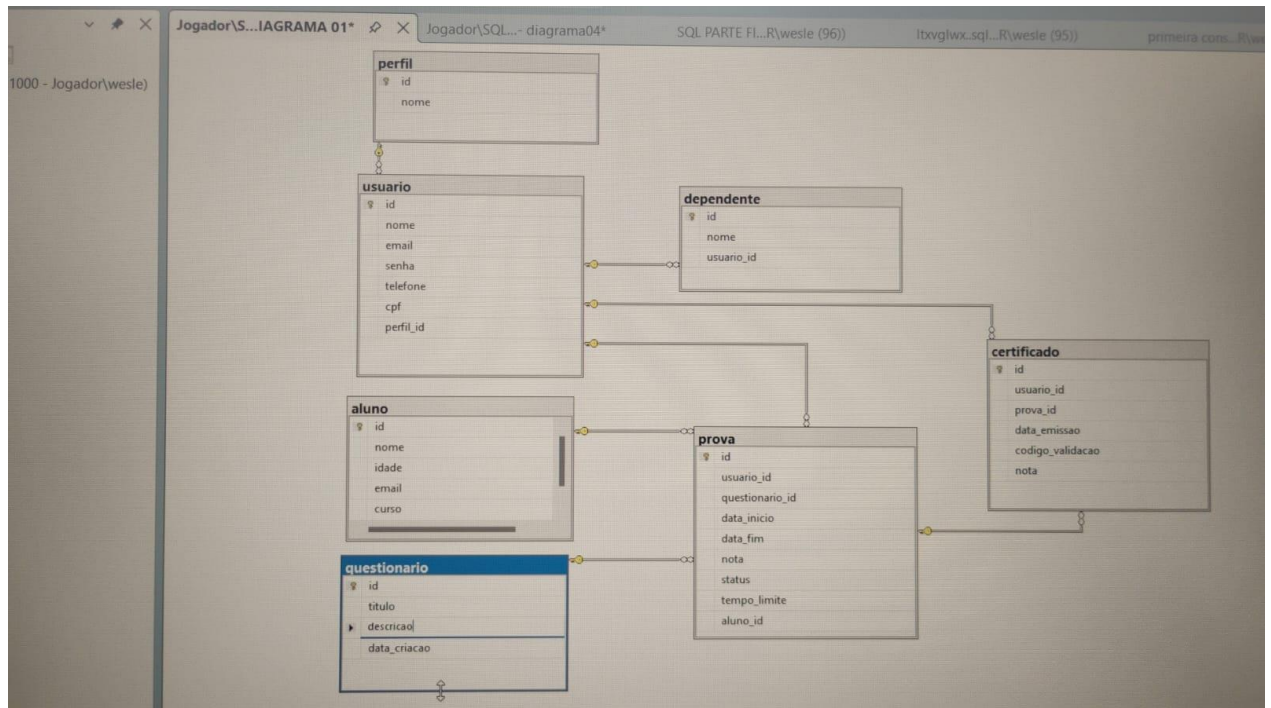
sd Sequence Diagram0

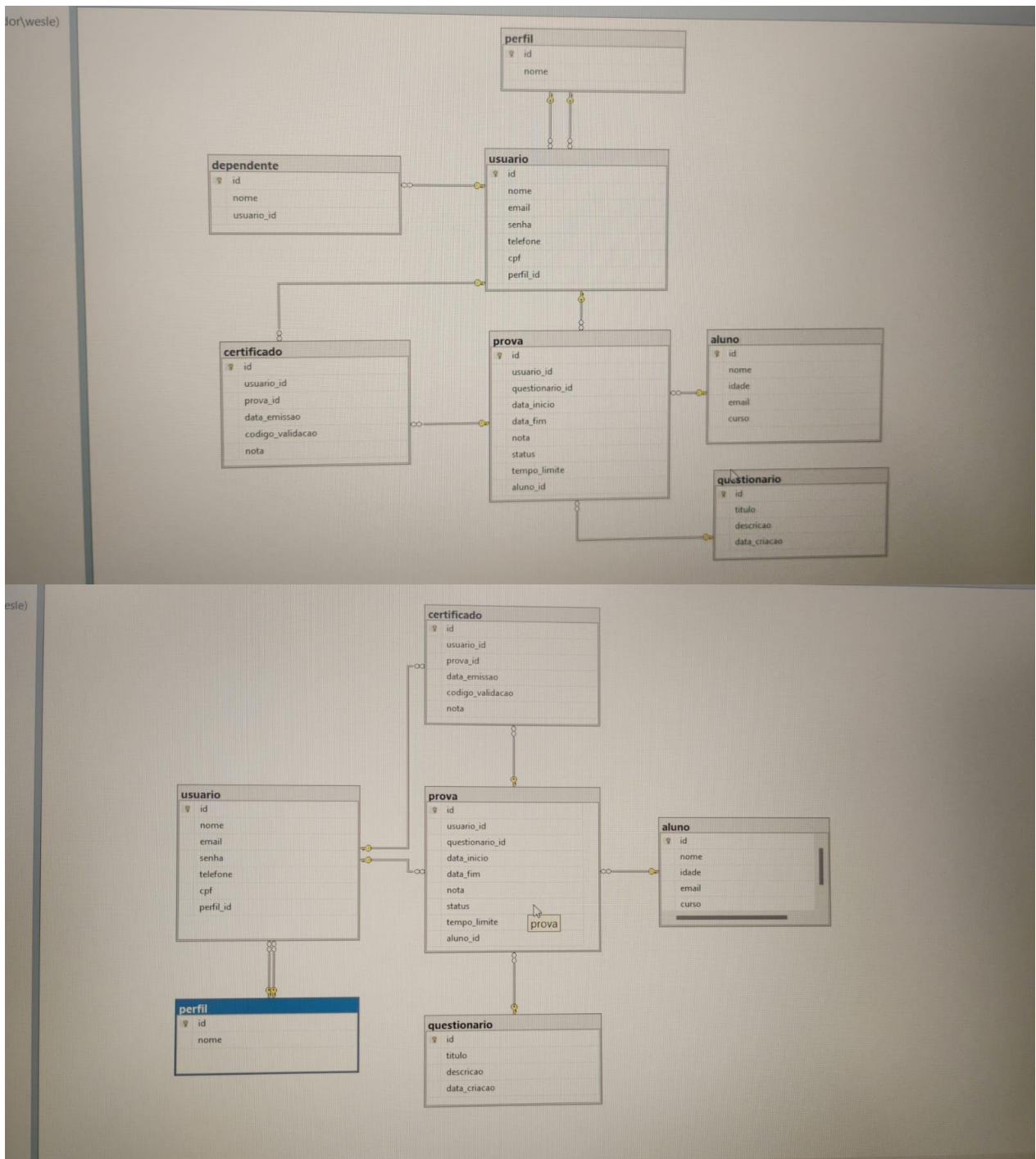


sd Sequence Diagram1



4 MODELAGEM E IMPLEMENTAÇÃO DO BANCO DE DADOS (SQL)





4.1 Entidades, Atributos e Lógica MER

O modelo entidade-relacionamento estruturado conta com chaves primárias e estrangeiras bem delineadas para garantir a integridade referencial:

- perfil (1) para usuario (N) — Permissões e segurança;

- usuario (1) para dependente (N) — Controle familiar/vínculos;
- usuario/aluno (1) para prova (N) — Registro de tentativas;
- questionario (1) para prova (N) — Relação entre banco de itens e avaliações;
- prova (1) para certificado (N) — Emissão vinculada ao êxito acadêmico.

4.2 Script de Implementação DDL

```

CREATE DATABASE EDUCA_TECH_SAAI;
GO
USE EDUCA_TECH_SAAI;
GO

CREATE TABLE perfil (
    id INT PRIMARY KEY IDENTITY(1,1),
    nome VARCHAR(100) NOT NULL
);

CREATE TABLE usuario (
    id INT PRIMARY KEY IDENTITY(1,1),
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    senha VARCHAR(255) NOT NULL,
    telefone VARCHAR(20),
    cpf VARCHAR(14) NOT NULL,
    perfil_id INT,
    FOREIGN KEY (perfil_id) REFERENCES perfil(id)
);

CREATE TABLE dependente (
    id INT PRIMARY KEY IDENTITY(1,1),
    nome VARCHAR(100) NOT NULL,
    usuario_id INT,
    FOREIGN KEY (usuario_id) REFERENCES usuario(id)
);

CREATE TABLE aluno (
    id INT PRIMARY KEY IDENTITY(1,1),
    nome VARCHAR(100) NOT NULL,
    idade INT,
    email VARCHAR(100) NOT NULL,
    curso VARCHAR(100) NOT NULL
);

CREATE TABLE questionario (
    id INT PRIMARY KEY IDENTITY(1,1),
    titulo VARCHAR(100) NOT NULL,
    descricao VARCHAR(255),

```

```

        data_criacao DATE
    );

CREATE TABLE prova (
    id INT PRIMARY KEY IDENTITY(1,1),
    usuario_id INT,
    questionario_id INT,
    data_inicio DATETIME,
    data_fim DATETIME,
    nota DECIMAL(5,2),
    status VARCHAR(50),
    tempo_limite INT,
    aluno_id INT,
    FOREIGN KEY (usuario_id) REFERENCES usuario(id),
    FOREIGN KEY (questionario_id) REFERENCES questionario(id),
    FOREIGN KEY (aluno_id) REFERENCES aluno(id)
);

CREATE TABLE certificado (
    id INT PRIMARY KEY IDENTITY(1,1),
    usuario_id INT,
    prova_id INT,
    data_emissao DATE,
    codigo_validacao VARCHAR(100),
    nota DECIMAL(5,2),
    FOREIGN KEY (usuario_id) REFERENCES usuario(id),
    FOREIGN KEY (prova_id) REFERENCES prova(id)
);

```

4.3 Consultas SQL (DML) para Auditoria

```

SELECT * FROM usuario;
SELECT * FROM prova;
SELECT * FROM certificado;
SELECT pergunta.titulo, categoria.nome
FROM pergunta
INNER JOIN categoria
ON pergunta.categoria_id = categoria.id;

```


5 INTERFACE WEB RESPONSIVA E PROTOTIPAÇÃO UX/UI

O design foi projetado para ser similar ao AVA da UNIP, priorizando a navegação lateral e cartões de disciplinas. A interface web atua como a camada visível, coletando e exibindo dados com fluidez.

5.1 Estrutura Semântica (HTML5) A organização do arquivo de marcação index.html utiliza tags estruturais bem delimitadas, o que otimiza a manutenção do código e a futura comunicação com a programação interna. O trecho a seguir ilustra a barra superior de navegação e o botão de acesso ao menu global:

```
<header class="global-nav" role="banner">
  <div class="gn-left">
    <button class="gn-hamburger" onclick="openHamburger()" aria-label="Abrir menu">
      <span></span><span></span><span></span><span></span>
    </button>
    <div class="nav-arrows" aria-label="Navegação de páginas">
      <button class="nav-arrow" onclick="navBack()" aria-label="Voltar página"></button>
      <button class="nav-arrow" onclick="navForward()" aria-label="Avançar página"></button>
    </div>
  </div>
  <button class="gn-logo logo-btn" onclick="switchScreen('home')" aria-label="Ir para a tela inicial">
    <!-- LOGO AQUI:  -->
    <span class="logo-text">Educa<span>Tech</span></span>
  </button>
  <div class="gn-right">${userBox}</div>
</header>;
```

5.2 Estilização e Design Tokens (CSS3)

A folha de estilos centraliza a identidade do projeto através de variáveis nativas CSS (:root). Isto assegura uniformidade às fontes carregadas (Nunito, DM Sans e Merriweather) e define as cores fixas das cinco disciplinas pedagógicas principais:

```

/* Tipografia */
--font-ui:      'DM Sans', 'Nunito', sans-serif;
--font-heading: 'Nunito', 'DM Sans', sans-serif;
--font-body:    'Merriweather', 'DM Sans', serif;

/* Paleta principal baseada em #7C3AED */
--color-primary:      #7C3AED;
--color-primary-dark:  #6D28D9;
--color-primary-darker: #5B21B6;
--color-primary-light: #A78BFA;
--color-primary-pale:  #F3F4F6;

/* Cores de apoio */
--color-accent:      #6D28D9;
--color-accent-dark: #4C1D95;

/* Neutros */
--color-bg:      #F3F4F6;
--color-surface: #FFFFFF;
--color-surface-2: #F3F4F6;
--color-border:  #DDD6FE;
--color-text:    #1F2937;
--color-text-secondary: #4B5563;
--color-text-muted: #9CA3AF;

/* Cores por curso */
--course-ads:      #4F8EF7; /* Análise e Desenvolvimento - Azul técnico */
--course-cc:       #7C5CBF; /* Ciência da Computação - Roxo profundo */
--course-design:   #F97316; /* Design Gráfico - Laranja criativo */
--course-ti:       #16A34A; /* T.I - Verde tecnologia */
--course-ec:       #2C0000; /* Engenharia da Computação - Vermelho escuro institucional */
--course-adm:      #0E7490; /* Administração - Teal corporativo */

```

5.3 Responsividade Dinâmica (Media Queries)

Para se ajustar perfeitamente tanto a telas de computadores quanto a dispositivos móveis, o layout utiliza Media Queries para redefinir o comportamento das grades e dos cards de forma fluida:

```

/* --- Tablet e maiores (min 600px) --- */
@media (min-width: 600px) {
  /* Grid de cursos com 2 colunas fixas */
  .courses-grid {
    grid-template-columns: repeat(2, 1fr);
  }

  /* --- Desktop (min 900px) --- */
  @media (min-width: 900px) {
    :root {
      --nav-height: 60px;
    }

    /* Grid de cursos 3 colunas */
    .courses-grid {
      grid-template-columns: repeat(3, 1fr);
    }
  }
}

```

5.4 Prototipação Interativa e Regras de Usabilidade

O modelo interativo construído nas ferramentas Figma e Stitch simula fielmente o comportamento mecânico que o software apresentará após sua programação completa, aplicando os princípios das Heurísticas de Usabilidade de Nielsen:

- **Feedback ao Usuário:** A interface mantém o usuário informado sobre seu status em tempo real através do preenchimento das barras de progresso parciais e da contagem regressiva contínua do cronômetro da tela de avaliações.
- **Prevenção de Erros:** Inclusão de janelas pop-up de confirmação (modais) antes da execução de ações críticas. O aluno é impedido de finalizar um teste ou deslogar da conta por cliques acidentais, exigindo uma validação consciente na interface.
- **Consistência Interna:** Todos os elementos visuais, como o arredondamento de cantos dos cartões, os estilos das fontes e o espaçamento entre blocos de texto, seguem um padrão idêntico em todas as páginas, reduzindo o esforço do aluno para aprender a navegar no sistema

6 Sistema Acadêmico

O protótipo desenvolvido tem como objetivo representar, de forma simplificada, um sistema acadêmico capaz de atender a diferentes perfis de usuário. A aplicação foi construída em linguagem C#, utilizando uma estrutura baseada em menus no console, o que facilita a demonstração didática das funções do sistema.

Além do código-fonte, foi utilizada modelagem em Astah para apoiar a análise estrutural do sistema. Essa etapa contribui para a visualização dos elementos principais do software, organizando entidades, responsabilidades e possíveis interações entre os módulos.

6.1 Objetivo do Sistema

O objetivo do protótipo é demonstrar um ambiente acadêmico básico no qual usuários com diferentes perfis possam acessar funções específicas. O administrador realiza cadastros e exclusões, o coordenador acompanha alunos e notas, o professor gerencia avaliações e o aluno consulta seu desempenho e realiza uma prova objetiva.5.4

6.1 Visão Geral de aplicação

A aplicação inicia com um menu principal que oferece acesso aos perfis Administrador, Coordenador, Professor e Aluno. Todos os dados são armazenados em memória por meio de uma lista estática de objetos da classe Aluno. Isso significa que, ao encerrar o programa, as informações cadastradas são perdidas, característica aceitável para um protótipo de validação.

6.1.1 Perfil de acesso

Perfil	Principais ações
Administrador	Cadastrar aluno, excluir aluno e visualizar notas gerais.

Coordenador	Lançar nota, zerar nota, consultar matéria, listar alunos e ver notas.
Professor	Adicionar nota, excluir nota e visualizar notas.
Aluno	Consultar a própria nota e realizar a prova completa.

6.2 FUNCIONALIDADES IMPLEMENTADAS

As funções centrais do ecossistema do protótipo abrangem os seguintes

requisitos operacionais:

- Cadastro de aluno por nome, com armazenamento em lista de objetos.
- Exclusão de aluno a partir da busca pelo nome informado.
- Lançamento e remoção de notas por perfis autorizados.
- Exibição do status da nota como 'Pendente'; quando a prova ainda não foi realizada.
- Realização de prova objetiva composta por três questões, com pontuação automática.
- Classificação final do aluno com base na nota obtida, exibindo aprovado ou Reprovado.

6.3 ESTRUTURA DE CLASSES

A estrutura do código foi definida com duas classes principais: Aluno e Program.

A classe Aluno funciona como entidade de dados, enquanto Program concentra a lógica de navegação, cadastro, consulta e avaliação.

6.3.1 Classe Aluno

Atributo	Tipo	Descrição
Nome	string	Armazena o nome do aluno.
Nota	int	Guarda a pontuação obtida na prova.
FezProva	bool	Indica se o aluno já realizou a prova.

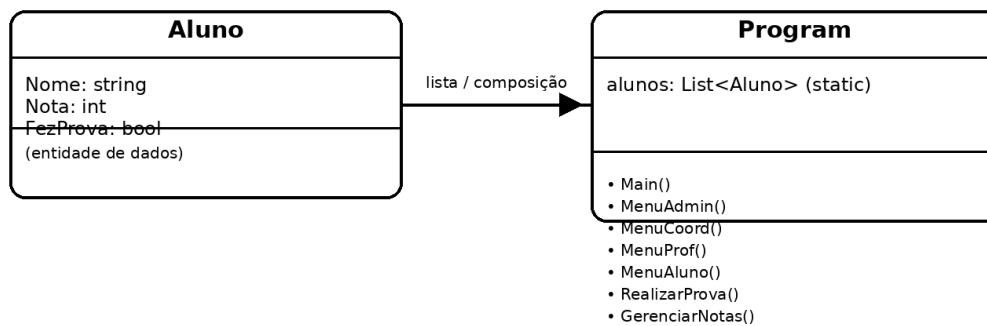
6.3.2 Classe program

Método / membro	Função
static List<Aluno> alunos	Lista em memória que armazena os alunos cadastrados.

Main()	Ponto de entrada do programa e exibição do menu principal.
MenuAdmin()	Executa ações do administrador.
MenuCoord()	Executa ações do coordenador.
MenuProf()	Executa ações do professor.
MenuAluno()	Executa ações do aluno.
RealizarProva(Aluno a)	Aplica a avaliação e calcula a nota final.
GerenciarNotas(bool remover)	Adiciona ou zera notas conforme o perfil.

6.3.3 Diagrama de Classes

Diagrama de Classes (reconstrução textual do código)

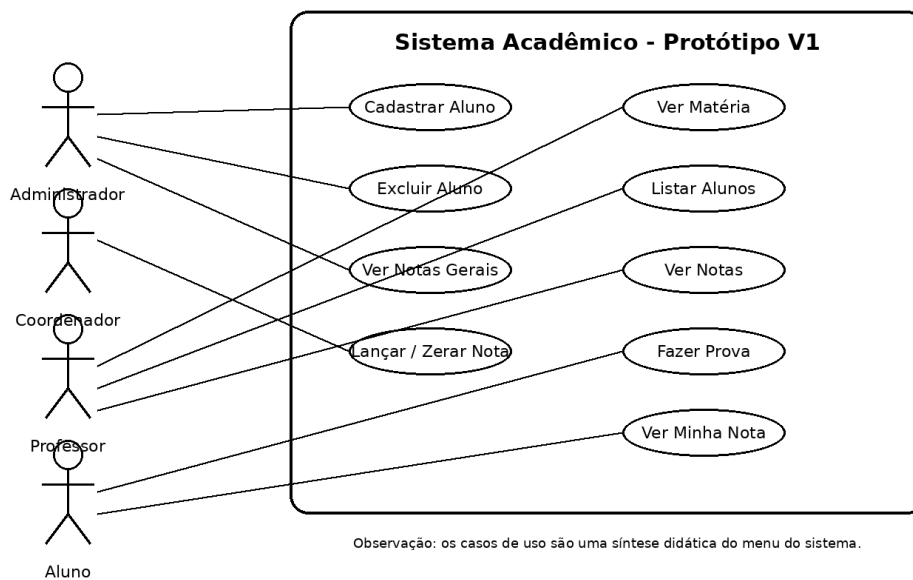


6.4 MODELAGEM NO ASTAH

O arquivo de modelagem enviado no formato do Astah foi utilizado como apoio para organizar o pensamento estrutural do sistema. A partir dos elementos identificáveis no modelo e do próprio código, é possível associar os atores e funções centrais do protótipo a um cenário acadêmico com diferentes permissões de uso.

Como síntese didática, o sistema pode ser representado por casos de uso ligados aos quatro perfis principais e às rotinas de cadastro, notas, consulta e prova.

Diagrama de Casos de Uso (baseado no protótipo)



6.5 REGRAS DE NEGÓCIO OBSERVADAS

As seguintes premissas operacionais guiam as validações do sistema:

- O aluno só consegue visualizar sua nota após realizar a prova.
- Quando a prova não foi feita, a nota aparece como 'Pendente';.
- A aprovação do aluno é exibida quando a nota é maior ou igual a 6.
- As notas podem ser atribuídas, alteradas ou zeradas por perfis internos.
- A prova contém questões objetivas com correção automática baseada na resposta digitada.

6.6 TECNOLOGIAS E RECURSOS UTILIZADOS

O desenvolvimento tecnológico utilizou as seguintes ferramentas:

- Linguagem C#.
- Console .NET para interação com o usuário.
- Classe List<T> para armazenamento temporário dos dados.
- Conceitos de orientação a objetos, como encapsulamento e composição de dados.
- Astah para modelagem UML e organização visual do sistema.

6.7 LIMITAÇÕES E POSSIBILIDADES DE EVOLUÇÃO

Por se tratar de um protótipo, o sistema ainda não possui persistência em banco

de dados, autenticação real de usuários, interface gráfica ou validação avançada de entradas. Essas características podem ser adicionadas em versões futuras para transformar o projeto em uma aplicação mais completa.

- Implementação de login e senha por perfil.
- Integração com banco de dados para salvar alunos e notas.
- Separação em camadas (model, service, repository e UI).
- Melhor tratamento de erros e validação de cadastro.
- Conversão da interface de console para Windows Forms, WPF ou Web.

7 CONCLUSÃO

O desenvolvimento do sistema SAAI cumpre com êxito os objetivos multidisciplinares do PIM III ao integrar de forma consistente Engenharia de Software, Modelagem UML, Banco de Dados, Programação Orientada a Objetos e Design UX/UI. A estrutura em HTML5/CSS3 foi validada e está totalmente adaptável para dispositivos móveis e desktops. O blueprint estrutural fornecido reduz expressivamente os riscos e erros de codificação, deixando a plataforma pronta para as integrações dinâmicas de retaguarda de processamento de notas e relatórios em tempo real.

REFERÊNCIAS

BRASIL. Política Nacional de Educação Ambiental – Lei nº 9.795/1999. Brasília, DF, 1999.

DIAS, Genebaldo Freire. Educação Ambiental: princípios e práticas. 9. ed. São Paulo: Gaia, 2010.

ELMASRI, Ramez; NAVATHE, Shamkant S. Sistemas de Banco de Dados. 7. ed. São Paulo: Pearson, 2018.

JACOBI, Pedro. Educação Ambiental, cidadania e sustentabilidade. Cadernos de Pesquisa, n. 118, p. 189-205, 2003.

MICROSOFT. Microsoft SQL Server Documentation. Disponível em: <<https://learn.microsoft.com/en-us/sql/>>. Acesso em: 2026.

NIELSEN, Jakob. Enhancing the Explanatory Power of Usability Heuristics. CHI '94 Proceedings, 1994.

UNIVERSIDADE PAULISTA – UNIP. Guia de normalização para apresentação de trabalhos acadêmicos da Universidade Paulista: ABNT. São Paulo: UNIP, 2025.